

Document Number:	P3416R0
Date:	2024-09-29
Target:	LEWG
Revises:	
Reply to:	Gor Nishanov (gorn@microsoft.com)

exception_ptr_cast: Add && = delete overload

An `exception_ptr_cast` is a facility offered by <https://isocpp.org/files/papers/P2927R2.html> to observe an exception stored in `exception_ptr`.

```
namespace std
{
    template<class T>
    const T* exception_ptr_cast(const exception_ptr& e) noexcept;
}
```

Such a function is dangerous if one passes a temporary `std::exception_ptr` directly to the `exception_ptr_cast`.

For example, given that `std::current_exception`

```
exception_ptr current_exception();
```

is allowed to return a copy of the currently handled exception and if it is the case, the object referenced by the `exception_ptr` will be destroyed when the `exception_ptr` is destroyed. Thus the following innocuous code

```
if (auto *ex = std::exception_ptr_cast<std::system_error>(std::current
    handle(ex->code); // <--- Use after free!!!
}
```

will result in **use after free** on Windows, as `std::current_exception` creates a copy of an exception, as the real one is allocated on the stack at the point of throw.

Delete rvalue overload

We can take an approach taken by several standard library facilities, such as:

```

template<class T> const T* addressof(const T&&) = delete;
template<class T> void as_const(const T&&) = delete;
template<class T> void ref(const T&&) = delete;
template<class T> void cref(const T&&) = delete;
// and others, like regex_iterators, etc.

```

that delete an overload taking rvalue reference to preventing accidental use of temporaries.

While we acknowledge that value category is not a lifetime, it is often strongly suggestive of one and thus allows us to eliminate an obvious mistake when composing `exception_ptr_cast` and `current_exception`.

Delete the overload will catch the misuse of `exception_ptr_cast` and `current_exception` and the user will have to correct the code by writing:

```

auto e = std::current_exception();
if (auto* ex = std::exception_ptr_cast<std::system_error>(e)) {
    handle(ex->code); // Safe, no temporary
}

```

Other alternatives considered

Another approach would be to modify the signature of `exception_ptr_cast` to take the `exception_ptr` by pointer:

```

template<class T>
const T* exception_ptr_cast(const exception_ptr* e) noexcept;

```

or by reference (like `any_cast`):

```

template<class T>
const T* exception_ptr_cast(exception_ptr& e) noexcept;

```

In both cases, we offer a less compelling interface, that raises questions, why do we take a pointer to `exception_ptr`? Does it make sense to pass null to this API? And, in the second case, does the API mutate the passed in `exception_ptr`?

The answer to all these questions is no, making the pointer/reference change is semantically unnecessary. We only changed the API to lessen the chance of obtaining a dangling pointer.

The approach of taking an `exception_ptr` by `const&` and delete-ing rvalue reference is addressing the problem directly without changing the API.

One argument against the deleting an overload that we will prevent some compelling code from working, however we are not aware of what such compelling code would be.

Another objecting would be that it can create a false sense of security and will not catch all cases of dangling.

While we agree that it won't catch all mistakes, it handles low-hanging fruit of misusing interacting with the most likely facility `exception_ptr_cast` will be interacting with, namely, `current_exception`.

Wording

Modify the wording offered by <https://isocpp.org/files/papers/P2927R2.html> by changing synopsis to:

```
exception_ptr current_exception() noexcept;
[[noreturn]] void rethrow_exception(exception_ptr p);
template <class E>
    const E* exception_ptr_cast(const exception_ptr& p) noexcept;
template <class E>
    void exception_ptr_cast(const exception_ptr&&) = delete;
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

Acknowledgments

Many thanks to Lewis Baker, who raised the concern and offered this approach to address it and to Arthur O'Dwyer who have written a thoughtful blogpost of why this is not a good idea.

References

<https://isocpp.org/files/papers/P2927R2.html> Inspecting `exception_ptr`

<https://github.com/scylladb/scylladb/blob/946d281/utils/exceptions.hh#L128-L151> ScyllaDb

<https://quuxplusone.github.io/blog/2024/07/03/dont-delete-const-refref/> Don't delete `&&`

<https://quuxplusone.github.io/blog/2019/03/11/value-category-is-not-lifetime/> Value Category is not lifetime

<https://abseil.io/tips/149> Tip of the Week #149: Object Lifetimes vs. `= delete`